HDOC Day-75

Implementation of Enumerations with Explicit Values and Menu-Driven Arithmetic Operations in C

Aim

To implement two C programs demonstrating enumerations: explicitly assigned integer constants and arithmetic menu choices using a switch statement.

Algorithm - Program 1

1. Define enum Values containing A = 10, B, and C.
2. Allow the compiler to assign consecutive values to B and C.
3. Print the values of A, B, and C.
4. Terminate the program.

Algorithm - Program 2

1. Define enum Operation with ADD = 1, SUBTRACT, and MULTIPLY.
2. Display the three menu choices and read the selected option.
3. Validate the choice and read two integers.
4. Use switch to perform addition, subtraction, or multiplication.
5. Display the result or an invalid-choice message.

C Program - 1: Explicit Enumeration Values

```
#include <stdio.h>

enum Values
{
    A = 10,
    B,
    C
};

int main()
{
    printf("Enumeration Values:\n\n");
    printf("A = %d\n", A);
    printf("B = %d\n", B);
    printf("C = %d\n", C);
    return 0;
}
```

C Program - 2: Arithmetic Operations Using Enumeration

```
#include <stdio.h>

enum Operation
{
    ADD = 1,
    SUBTRACT,
    MULTIPLY
};

int main()
{
    enum Operation choice;
    int option, num1, num2, result;

    printf("----- ARITHMETIC OPERATIONS MENU -----\n");
    printf("1. ADD\n");
    printf("2. SUBTRACT\n");
    printf("3. MULTIPLY\n");
    printf("Enter your choice: ");
    scanf("%d", &option);

    if (option < ADD || option > MULTIPLY)
    {
        printf("Invalid choice!\n");
        return 0;
    }

    choice = (enum Operation)option;
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);

    switch (choice)
    {
        case ADD:
            result = num1 + num2;
            printf("Result: %d\n", result);
            break;
        case SUBTRACT:
            result = num1 - num2;
            printf("Result: %d\n", result);
            break;
        case MULTIPLY:
            result = num1 * num2;
            printf("Result: %d\n", result);
            break;
        default:
            printf("Invalid operation!\n");
    }
    return 0;
}
```

Note: These programs assume valid integer inputs and arithmetic results within the range of int.

Explanation

An enumeration (enum) is a user-defined type whose named constants have integer values. If a constant is assigned a value, subsequent unassigned constants increase by one. Thus, A = 10 makes B = 11 and C = 12.

The second program uses ADD = 1, SUBTRACT = 2, and MULTIPLY = 3. The selected numeric choice is converted to enum Operation, then switch executes the corresponding arithmetic operation. The break statements prevent fall-through.

For example, ADD with 10 and 20 calculates 10 + 20 = 30. Named constants improve readability and make the menu logic easier to maintain.

Enumeration Reference Tables

Constant	Integer value
A	10
B	11
C	12

Operation	Value	Action
ADD	1	Addition
SUBTRACT	2	Subtraction
MULTIPLY	3	Multiplication

Test Case Table

Test	Program / Input	Expected output
1	Explicit enum / no input	A = 10, B = 11, C = 12
2	ADD 10 20	Result: 30
3	SUBTRACT 20 10	Result: 10
4	MULTIPLY 5 6	Result: 30
5	ADD -5 10	Result: 5
6	SUBTRACT 5 10	Result: -5
7	MULTIPLY 0 10	Result: 0
8	Choice = 4	Invalid choice!

Sample Outputs

Program 1 - Explicit values

```
Enumeration Values:

A = 10
B = 11
C = 12
```

Addition

```
----- ARITHMETIC OPERATIONS MENU -----
1. ADD
2. SUBTRACT
3. MULTIPLY
Enter your choice: 1
Enter two integers: 10 20
Result: 30
```

Subtraction

```
----- ARITHMETIC OPERATIONS MENU -----
1. ADD
2. SUBTRACT
3. MULTIPLY
Enter your choice: 2
Enter two integers: 20 10
Result: 10
```

Multiplication

```
----- ARITHMETIC OPERATIONS MENU -----
1. ADD
2. SUBTRACT
3. MULTIPLY
Enter your choice: 3
Enter two integers: 5 6
Result: 30
```

Invalid choice

```
----- ARITHMETIC OPERATIONS MENU -----
1. ADD
2. SUBTRACT
3. MULTIPLY
Enter your choice: 4
Invalid choice!
```

Result

The first C program demonstrates enumeration constants starting at A = 10, followed by B = 11 and C = 12. The second program represents addition, subtraction, and multiplication with enum constants and uses switch to perform the selected operation. Both programs demonstrate clear, organized use of enumerations in C.